When Abstraction Becomes Indirection:

C against C++ when the maintainer is an agent.
The control arm, the instrument, and the prediction

Vasili Gavrilov

23 August 2026

Abstract

Abstraction hides detail because a person can hold only so much at once. An agent has no such limit, and pays instead for what it must reconstruct. This paper asks whether the trade still holds when the maintainer is an agent, and reports what can be measured before the treatment arm exists.

Over public code by other authors: C++ carries 60.9 functions per thousand code lines against C's 28.4, over 430,081 and 1,214,580 lines, and the median C++ function has cyclomatic complexity 1, straight-line code that calls something else. The highest-complexity function in a project is nearly always the code facing a command line, an HTTP request, a wire format, an instruction stream or a file: 20 of the 22 projects here that carry a function above complexity 10, the two exceptions optimised interior kernels. That maximum is a property of a program's boundary, not a verdict on how it was written.

Over one author's code: explicit procedural C carries 1.3 to 2.7 times the lexical tokens of the same behaviour in his Java; lexical count and tokenizer count agree only at ordinary writing density, so which count a comparison comes with decides which language it calls compact; and a production Java service of 39,621 lines reaches SQL through three artifacts, with no entity, DTO, mapper or framework across 37 tables, which shows the seven-artifact path is not required to deliver that behaviour.

The agent measurement is a null with a stated cause. In 60 hypothesis-blind runs on maintenance tasks judged by hidden tests, all passed, the source under maintenance was at most 2.7% of context at the median, and a run cost a 30.9k-token harness constant plus 926 a turn: five languages tied within the spread of three repetitions. Retrieval was one file read, so representation had no opportunity to cost anything. Nothing under 15% of median context is resolvable at that repetition count.

No C++ and no framework implementation of any behaviour used here exists, so the comparison the paper is built for is absent. What is present is the instrument, at $0.11 and 29 s a run, the calibration above, and a prediction with its refutations recorded before the code is written.

1 Question

A class hides a representation, an interface hides an implementation, a container hides memory, a framework hides control flow. Each is a remedy for a limit of human working memory. An agent is not under that limit and is under others: the context it carries, the retrieval it performs, and the relationships it resolves before acting. A hidden implementation is not read for free by an agent; it is a relationship to resolve, which abstraction adds to rather than removes. This paper keeps the three costs apart:

1. **Representation**: what is in the repository. Lines, tokens, identifiers, functions, call edges.
2. **Reconstruction**: the relationships an agent must resolve to establish the behaviour a task touches.
3. **Maintenance cost**: context tokens, wall time, turns, failures, hidden-test pass.

The question is whether the first predicts the third, or whether the second has to be modelled to predict it.

The axis is the fraction of a program's behaviour that is written down in the program. C sits at the low end, close to a portable assembler: a call goes where the source says it goes. C++ is the first step off it and the cleanest to measure, sharing a machine model, a compilation model, a performance envelope

and a library lineage with C, so a C to C++ pair varies the human-facing layer and holds the rest fixed. A framework is the far end of the same axis: an annotation, a naming convention, a dependency container or a cluster manifest decides what runs, and the code acting on it is not in the repository in any form.

Ada, Rust, Java and Python are not on this axis. Each changes the runtime, the memory model and the library ecosystem at once, so a ratio against any of them measures a bundle. They are here as controls, calibrating the effect a language change produces when the change is not the layer under study. The rungs that are on the axis have to be written, and Section 9 specifies them.

The answer is not obvious in either direction. Hiding a representation adds a relationship to resolve. Hiding a lifetime removes bookkeeping that would otherwise be carried correctly through every path. RAII and the standard containers are where abstraction plausibly lowers reconstruction cost.

2 What is withdrawn

An earlier draft reported C at 292 physical lines and 1,863 lexical tokens against C++17 at 218 and 1,369 for the key and posting subsystem of `ais`, the C++ 26.5% smaller. No tokenizer, file list or commit was recorded, and no C++ implementation of that subsystem exists in any of the author's repositories [1]; the only C++ in `ais` is the Flutter runner the toolchain generates. The C side at `84e3ad5` is `key.c`, `key.h`, `post.c`, `post.h`: 397 physical lines, 266 code, 84 comment, 1,882 lexical tokens, 521 identifier tokens, 10 functions. The earlier C figure is close to the two `.c` files alone at some earlier commit. The C++ figure has no source and is withdrawn. Nothing below depends on it.

3 Material

Every repository is public under `https://github.com/Anode1`. Table 1 lists what was measured and at which commit.

Repository	Language	Commit	What it is
`ais`	C	84e3ad5	associative index; the five-language bench in `tests/perf`
`cjitter`	C	4aaee2c	four stochastic searches and a uniform control
`bpnn`	C	7640917	backpropagation network and a tabular predictor
`linearr`	C, Java	0a79370	streaming least squares; `java/` is the fit, line for line
`iac`	C	20fb0a3	inter-agent dispatcher
`graphcrawl`	C, JS	ef5b405	depth-bounded graph navigator over a sorted text file
`mincdp`	C, Java	1bebbe3	Chrome DevTools Protocol client, one file per language
`ljms`	Java	6677246	a work queue that is one database table
`aisgedcom`	Java	ec0cbac	GEDCOM genealogy toolkit, written 2009 and 2011
`aisconvert`	Java	d3bc723	personal genomics toolkit, written 2009
System A	Java	ef0f7e0d	a commercial web application in production

Table 1: Repositories measured. The years for `aisgedcom` and `aisconvert` are their file headers': both were imported to git in 2015 and no Java file has changed since.

3.1 Three equivalent sets, the control arm

None varies the abstraction layer on its own; each changes the runtime and the library ecosystem with it. They fix the baseline, the fixtures and the noise floor.

bench. `ais/tests/perf` holds one program in five languages: a scan of a store counting lines containing a substring, and the two-pointer intersection of two sorted posting lists. Its `LANG_COMPARISON` note [4] checks that all five agree on a 1,000,000-record index (379,090 matching records, 21,923 common ids); this paper's hidden fixtures (Section 7) add six more, including empty, identical and disjoint lists and an overlap at the last element, and all five pass. One boundary: the Python idiomatic scan path is

`data.count(pat)`, counting occurrences where the other four count lines, so on a line holding the substring twice Python differs. The fixtures stay where the five agree, at most one occurrence per line.

mincdp. A header-only C client (`cdp.h`, `demo.c`) and a Java client (`Cdp.java`, `Demo.java`) with the same command set, each with a demo driving the same page and printing the same assertions. Equivalence rests on that demo, which was not re-run here.

linearr. By their own headers `Linearr.java` is the twin of `main.c` with `process.c`, `Regress.java` of `regress.c` line for line, and `Csv.java` of `csv.c`. `make java` fits every example with both and diffs the coefficient tables. The C `main.c` carries a scoring mode the Java lacks, so this pair's C/Java ratio is an upper bound.

3.2 Measurement procedure

- *physical*: newline count of the file. *code*, *comment*: cloc 1.98 [8].
- *lexical tokens*: the count from a lexer in `measure.py` over the comment-stripped source. Identifiers, numbers, string and character literals, multi-character operators and single punctuation are one token each. Python is tokenized by the standard library's `tokenize`, with NEWLINE, INDENT and DEDENT counted (they are Python's braces and semicolons) and NL and COMMENT dropped.
- *identifier tokens*: identifier tokens that are not keywords of the language. *distinct*: the number of different ones.
- *o200k*: tiktoken 0.14.0 [10], encoding `o200k_base`, over the raw file and over the comment-stripped file. Anthropic's tokenizer is not public; o200k is a proxy, and `cl100k_base` is in the JSON output and differs from it by under 4% on every set.
- *functions, NLOC, CCN*: lizard 1.24.0 [9]: functions found, their non-blank non-comment lines, their cyclomatic complexity. lizard does not parse Ada.
- *nesting*: the deepest brace nesting; for Python the deepest indent; for Ada the deepest block (`begin`, `loop`, `if`, `case`, `record`, `select`).
- *call graph* (C only): libclang 18.1.1 parses every `.c` with `-std=c99` and the project's include directories. Nodes are functions defined in the set; edges are calls between them; the longest chain is the longest acyclic path after cutting back edges in a depth-first search. Zero parse errors on every project.
- *fn macros*: `#define` with a parameter list.

Exclusions: `tests.c` in each C project (a unit-test file), `graphcrawl/c/web.c` (414 KB generated from `web/` by `embed.sh`).

4 Source measurements

								o200k			NLOC		CCN	
Set	Files	Phys.	Code	Cmt.	Lexical	Ident.	Distinct	raw	strip	Fns	med	max	med	max
bench/C	1	31	31	0	863	278	69	774	774	6	3.5	8	4.5	11
bench/Ada	1	133	127	0	1,071	359	82	1,373	1,373	n/a	n/a	n/a	n/a	n/a
bench/Rust	1	73	66	3	609	172	53	739	668	4	16	21	5	9
bench/Java	1	66	56	5	682	195	51	824	709	4	12	19	7	11
bench/Python	1	25	25	0	496	140	41	478	478	2	5	8	3.5	6
mincdp/C	2	540	419	64	4,467	1,197	185	6,932	5,743	24	12	51	5.5	25
mincdp/Java	2	213	151	36	1,682	510	151	2,615	2,032	18	5.5	23	2	6
linearr/C	7	2,135	1,525	396	12,179	3,480	428	25,526	17,918	66	6	217	2	73
linearr/Java	3	737	483	201	4,692	1,373	171	9,017	5,666	15	7	217	3	74

Table 2: The equivalent sets. lizard does not parse Ada. Brace or indent nesting is 3, 7, 4, 4, 3 for the bench in file order, 4 and 5 for `mincdp`, 6 and 6 for `linearr`. The bench C packs several statements per line (31 physical lines, 863 tokens), so its per-function NLOC is not comparable with the others.

4.1 What the ratios say

C over the other language, same job:

Pair	Lexical	o200k raw	Code lines
bench C / Ada	0.81	0.56	0.24
bench C / Rust	1.42	1.05	0.47
bench C / Java	1.27	0.94	0.55
bench C / Python	1.74	1.62	1.24
mincdp C / Java	2.66	2.65	2.77
linearr C / Java (upper bound)	2.60	2.83	3.16

The author's C is the larger representation of the same job by 1.3 to 2.7 against his Java, and larger than Rust and Python on the bench. Only Ada is larger than the C.

The two token measures diverge on the bench and agree on the two programs. The bench C packs 863 tokens into 31 physical lines, 28 per line, and o200k rewards dense text: it costs 0.94 of the Java and 1.05 of the Rust in o200k while carrying 1.27 and 1.42 times their lexical tokens. On `mincdp` and `linearr`, at ordinary density, the two ratios agree to within a hundredth and a quarter. An agent pays the tokenizer's count, and that count tracks lexical tokens only at ordinary density. Line counts are the worst of the three: the bench C has a quarter of the Ada's code lines and four fifths of its lexical tokens.

4.2 The C style, in numbers

The `ais` style guide [3] asks for small functions, explicit state, minimal preprocessing and shallow module relationships. Table 3 measures the seven C projects (20,055 code lines), the author's Java (10,051), and 39,621 code lines of a commercial project, against those claims. Test packages are excluded throughout, as are three third-party files carried in the two genomics toolkits (Sun's `SwingWorker`, twice, and SourcePortal's `UCProperties`).

					Function NLOC			CCN						Call graph			
Project	Files	Code	Lexical	Fns	med	p90	max	med	p90	max	>10	Nest	Macros	Edges	Fan-out med	max	Chain
ais/c	40	10,597	82,135	438	13	38	601	5	14	253	92	8	11	802	1	55	18
cjitter/c	4	603	5,650	28	9	34	99	3	18	38	6	5	1	47	1	7	9
bpnn/c	26	3,224	30,007	123	9	53	307	3	20	141	20	6	2	207	0	30	6
linearr/c	25	2,956	24,656	133	7	43	217	3	15	73	20	6	11	239	1	34	9
iac	16	865	8,314	41	14	43	56	6	21	33	9	7	0	99	1	15	6
graphcrawl/c	11	1,238	8,709	54	14	42	108	6	13	40	10	4	0	93	1	14	10
mincdp/c	3	572	5,762	27	15	38	71	6	19	25	6	4	0	45	1.5	13	7
ljms/src	6	561	3,891	41	10	23	37	3	6	9	0	6					
aisgedcom/src	65	4,620	33,308	236	7	36	211	2	7	41	15	9					
aisconvert/src	52	4,236	29,357	225	9	37	133	2	9	44	17	9					
linearr/java	3	483	4,692	15	7	70	217	3	43	74	4	6					
mincdp/java	2	151	1,682	18	5.5	13	23	2	5	6	0	5					
System A	121	39,621	334,615	1,485	12	51	1,249	4	14	329	244	10					

Table 3: The C corpus, the author's Java, and System A (production sources, tests excluded). Fan-out is calls from a function to other functions of the project; chain is the longest acyclic call path. Blank cells: not measured for Java.

The median claim holds: median function 7 to 15 lines, median CCN 3 to 6, the median function calling one other function of the project or none. The tail claim does not. `main` in `ais/c/main.c` is 601 lines at CCN 253, `handle` in `serve.c` 423 and 169, `feed_import_from_map` 302 and 111, `fit_stream` in `bpnn` 307 and 141. In `ais` 92 of 438 functions exceed CCN 10, and the longest call chain is 18 functions.

Ports carry the C's shape: `linearr/java` has its C's maximum function line for line (217 lines, CCN 73 and 74). Java written without a C original does not. `aisgedcom` and `aisconvert`, 8,856

code lines and 461 methods from 2009 and 2011, hold the median at 7 and 9 lines and CCN 2 and cut the tail to 32 of 461 methods over CCN 10 (7%, against 21% in `ais`) with maxima of CCN 41 and 44. `ljms` has nothing above CCN 9.

Table 4 names the maximum function of every project. In all thirteen it stands at a boundary: a command line, an HTTP request, a WebSocket frame, a file being streamed, a library's exported entry. The one project whose maximum is a leaf, `mincdp/java` at CCN 6, has no tail to place. Section 6 tests this outside the author's code, where it holds in nine of eleven.

Project	Highest-complexity function	The boundary it stands at	NLOC	CCN
ais/c	`main`	command line	601	253
cjitter/c	`cjitter_compare_masked`	the library's exported entry	99	38
bpnn/c	`fit_stream`	a file streamed record by record	307	141
linearr/c	`main`	command line	144	73
iac	`main`	command line	47	33
graphcrawl/c	`main`	command line	108	40
mincdp/c	`cdp_ws_read_msg`	a WebSocket frame	34	25
ljms/src	`Processor.start`	the queue poll loop	37	9
aisgedcom/src	`ArgsParser.setArgs`	command line	120	41
aisconvert/src	`ArgsParser.setArgs`	command line	133	44
linearr/java	`Linearr.main`	command line	217	74
mincdp/java	`Demo.isPng`	none: a leaf, and the project has no tail	8	6
private/java	`doGet`	an HTTP request	1,249	329

Table 4: The function of highest cyclomatic complexity in each project, and what it faces. Twelve of the thirteen stand at a boundary; the thirteenth has no function above CCN 10.

Its size varies with how much work it does before delegating. The two toolkits' parser sets properties and returns, the work following a call later from a 57-line `Main`: CCN 41 and 44. `ais`'s `main` parses the options and then runs a second `switch` of 28 cases executing each command in place: CCN 253. System A's `doGet` and `doPost` decode a request and serve it in one method: CCN 329 and 300. Same boundary, same author's habits, a factor of eight. System A is otherwise the author's C profile in another language: median method 12 lines at CCN 4, p90 51 and 14, against `ais`'s 13, 5, 38 and 14, with 244 of 1,485 methods over CCN 10 (16% against 21%).

The median is a property of how the code is written and is the same everywhere here. The maximum is a property of where the boundary is and how much it does. An experiment on "explicit C" conditions on both, and the phrase names only the first.

5 System A: a case report

System A is a commercial web application in production. Its name, its domain and its business logic are withheld at the owner's instruction. Its structure is reported, on the convention of a clinical case report: the subject is anonymous, every measurement that does not identify it is given in full, and the conclusion is bounded to what one case can carry. The author is one of its developers and has full repository access. The numbers come from the same `measure.py` as the rest of this paper, run against the private tree.

Production Java	121 files, 39,621 code lines, 1,485 methods
Tests	129 files, 47,709 code lines
Front end	46,390 lines of JavaScript, no framework, no build step
Relational schema	37 tables
Persistence layer	39 classes, 25,772 lines, 679 methods
SQL	622 statements, all written in the source, 509 prepared
Methods returning the response body directly	205
Entity, DTO, mapper and repository classes	0
Data-carrier classes	1, of 209 lines
Imports of Spring, Jackson, JPA, Hibernate or MapStruct	0
Deployed runtime	8 jars, 3.3 MB, of which the JSON layer is 24 KB
Path from HTTP to SQL	3 artifacts
Steps on that path with no file to open	0

Table 5: System A, measured. Tests are excluded from the production figures throughout this paper.

A `ResultSet` becomes the response body in the method that ran the query. No object graph stands between the database and the client, and nothing decides at runtime that is not written down.

The case establishes a negative. The seven-artifact path of Table 10 is not required to deliver this class of behaviour, because a system delivering it without them is in production. A case report carries that and nothing more: it does not show the short path is cheaper, safer or more common, and one case supports no claim about frequency. The comparison that would is the framework rung of Section 9, which does not exist.

The low end of the axis is therefore available in Java at 39,621 production lines, which makes it an architectural choice rather than a property of C. Its function profile is the author's C profile in another language, and its maximum sits at the boundary like every other maximum here.

6 Other people's code

The same script, over eleven public projects by other people: five C systems and six C++ libraries, 1.64 million code lines. It tests two of the paper's claims outside the author's hands and fixes the feature rates the C++ conditions of Section 9 are built at.

Each project contributes its main implementation directory. Tests, vendored third-party code, benchmarks, examples and fuzzers are excluded by path, and 130 generated files by name or by a generation marker in their first 4 kB. That exclusion is not cosmetic: before it, the highest-complexity function in `postgres` was a Snowball-generated stemmer and the deepest nesting in `protobuf` was its own compiler's output.

Project	Commit	Files	Code	Fns	med NLOC	p90 NLOC	med CCN	p90 CCN	max
sqlite	d6bae5e	144	129,203	3,971	14	53	3	15	1,330
redis	6331fbe	214	140,575	5,815	11	43	3	12	326
nginx	3f6f782	403	179,735	3,236	27	91	5	20	190
curl	c2676bf	384	125,481	3,825	16	60	4	17	90
postgres	29636b4	848	639,586	17,664	17	72	3	14	258
leveldb	7ee830d	89	11,197	857	5	22	1	5	33
rocksdb	e6a2ee0	559	162,453	9,516	5	30	1	6	163
protobuf	672eda6	561	202,746	11,913	7	30	1	5	103
fmt	e27cc20	20	13,783	1,201	4	13	1	4	77
spdlog	f5f173a	107	20,742	1,833	4	13	1	4	77
json	734fd30	47	19,160	853	5	29	1	10	273

Table 6: Eleven public projects, five in C and six in C++, generated files excluded.

6.1 The indirection is visible before an agent runs

C++ carries 60.9 functions per thousand code lines against C's 28.4, a factor of 2.1, over 430,081 and 1,214,580 lines respectively. The median C++ function is 4 to 7 lines at cyclomatic complexity 1, against 11 to 27 lines at complexity 3 to 5 in C. Complexity 1 at the median means the median C++ function contains no branch at all: it is straight-line code that calls something else.

That is this paper's axis, in released code by other authors, without an agent and without a hypothesis. The path an agent walks is longer because there are twice as many functions per unit of behaviour, and the tokens it reads to reach the point of change are spread over more artifacts. None of it settles the cost, which Section 9 is for. It establishes that the thing whose cost is in question is real, large and not peculiar to the author.

6.2 Where the maximum sits, tested outside

In the author's thirteen projects every maximum above complexity 10 stood at a boundary. Over these eleven that holds nine times: `sqlite3VdbeExec` is the bytecode interpreter's opcode dispatch, `rdbLoadObject` decodes a database file format, the nginx request-line parser handles HTTP, `ExplainNode` walks a plan tree to produce output, `InterpretArgument` parses a command line, `parse_chrono_format` reads a format string in both `fmt` and `spdlog`, a MessagePack reader decodes a wire format, and `curl_easy_strerror` maps internal codes to user-facing strings at the API surface.

It fails twice, both the same kind. `leveldb`'s `DoCompactionWork` at CCN 33 and `rocksdb`'s `crc32c_3way` at CCN 163 are interior: a compaction driver and a hand-unrolled checksum kernel. Optimised inner loops accumulate branches for reasons unrelated to a program's edges.

The surviving claim is weaker than the author's own code suggested, and is stated at its measured strength: the highest-complexity function in a project is usually the code facing the outside, and the exceptions here are optimised kernels. Twenty of the twenty-two projects across both corpora that carry a tail at all. Reading that number as a verdict on a language or a style guide remains wrong, which is the use this paper objects to.

6.3 Feature rates for the C++ conditions

Table 7 gives the counts the pairs of Section 9 are parameterised by, so the C++ side is built at rates that occur rather than rates that favour the hypothesis. Counts are lexical, over comment-stripped source; a semantic count would need each project's build configuration.

Project	Code	`virtual`	`override`	`template`	`operator`	inherits	smart ptr	lambda
leveldb	11,197	7.2	18.1	2.4	4.1	3.5	0.0	0.0
rocksdb	162,453	9.8	14.1	2.5	3.1	2.4	13.1	1.6
protobuf	202,746	1.5	6.1	10.0	3.1	1.8	2.3	2.2
fmt	13,783	0.1	0.3	67.5	7.1	1.9	0.5	1.7
spdlog	20,742	1.1	5.9	57.6	6.3	5.1	12.2	1.6
json	19,160	0.9	0.4	32.5	11.6	0.7	0.3	1.8
All	430,081	4.7	9.0	12.1	3.8	2.2	6.6	1.9

Table 7: C++ feature occurrences per 1,000 code lines, counted lexically.

The variance is larger than the mean and it is structured. `fmt` and `spdlog` carry 67.5 and 57.6 template declarations per thousand lines as header-only generic libraries; `leveldb` carries 2.4. Virtual dispatch runs the other way, 9.8 and 7.2 in `rocksdb` and `leveldb` against 0.1 in `fmt`. Library C++ and systems C++ use different halves of the language, so the mixed condition of Section 9 is built at systems rates, that being what an `ais` or a System A would become, and the paper reports the choice.

The corpus is six libraries and no applications. C++ applications are under-represented on public hosts, and their mix is likely different again; these rates are library and systems C++, not C++ in general.

7 Agent pilot: the control arm

7.1 Protocol

The five bench implementations are the five conditions. Each run copies one into a fresh directory holding only the source file, a neutral `README.md` with build and run commands, `check.sh` (builds and runs both operations on the visible fixtures), and `fixtures/` (a 24-line store and two posting lists of nine and seven elements). The directory is a fresh git repository; trace, diff and verdict are written outside it.

Four tasks, one text each, identical for every language, naming behaviour and output but never a file, a function or a language feature:

T1 add a `diff` operation: ids in the first list absent from the second, one forward pass, no set or map, rows beginning `diff` with `only=<count>`;

T2 make the scan case-insensitive for ASCII letters, every other byte exact, each line counted at most once;

T3 the intersection reports a wrong count (4 is correct for the fixtures); find and fix it. The defect is one substitution per language: the merge advances the wrong side when heads differ, yielding 0;

T4 after the intersection's rows, print `first` and `last` for the common ids, or `none` for both.

Each task ends: keep everything else unchanged, verify with `check.sh`, do not ask questions.

The agent is Claude Code 2.1.241 headless, model `claude-sonnet-5`, default system prompt plus one sentence (work only in the current directory, do not ask questions), default tools, no MCP servers, no user settings or memory, at most 50 turns and $4 a run. It is told nothing about the study, the other languages or the hidden tests. The 60 runs (4 tasks $\times$ 5 languages $\times$ 3 repetitions) ran in seeded random order, four at a time.

Hidden tests, never in the run directory, judge each result: the program is rebuilt from a clean tree with the README's command and run on a 400-line store whose last line has no newline, and on five posting-list pairs (random with overlap, identical, disjoint, one empty, overlap only at the last element). Every printed row must carry the right count, Python printing two rows per operation and both having to be right, and the task's own output must be right on every pair. All five originals pass the non-task checks; all five bugged copies fail the intersection.

Per run, from the stream-json trace: context tokens (the sum over API calls of input, cache-read and cache-creation tokens, what the model was shown cumulatively), output tokens, turns, tool calls by tool, files read, shell commands, edits, wall time, cost, hidden-test verdict, final diff.

7.2 Results

All 60 runs passed the hidden tests, and every run stopped of its own accord: no turn or budget cap was reached. Table 8 gives each cell, Table 9 each language over the four tasks.

Task	Lang.	Pass	Context med.	Context range	Output median	Turns	Tool calls	Wall s
T1 add diff	C	3/3	204,558	203,656–204,778	3,049	10	9	31
	Ada	3/3	249,748	246,739–324,890	3,413	12	11	36
	Rust	3/3	274,313	274,294–308,837	2,727	10	9	32
	Java	3/3	206,650	204,276–311,586	3,310	11	10	34
	Python	3/3	236,730	200,866–240,527	2,812	9	8	30
T2 case-insens. scan	C	3/3	200,517	200,092–233,574	1,937	8	7	24
	Ada	3/3	207,292	205,749–207,582	2,793	8	7	31
	Rust	3/3	200,856	200,449–235,943	2,155	8	7	27
	Java	3/3	199,965	199,910–201,120	1,804	8	7	27
	Python	3/3	240,090	203,900–240,415	4,027	9	8	47
T3 repair merge	C	3/3	165,743	164,868–165,790	1,874	9	8	23
	Ada	3/3	202,332	167,680–202,440	1,017	6	5	17
	Rust	3/3	164,745	164,491–197,106	1,021	7	6	18
	Java	3/3	198,952	198,075–201,535	1,321	7	6	23
	Python	3/3	196,857	163,456–229,857	1,266	7	6	22
T4 first/last id	C	3/3	201,478	200,580–235,736	2,263	8	7	30
	Ada	3/3	282,105	245,366–283,467	3,156	10	9	35
	Rust	3/3	200,245	199,778–200,338	1,973	8	7	26
	Java	3/3	200,068	199,967–200,331	1,880	8	7	26
	Python	3/3	231,944	163,394–233,382	1,665	8	7	26

Table 8: Each cell of the pilot: three runs. Context is the cumulative prompt over the run's API calls; turns, tool calls and wall time are medians.

Language	Pass	Context median	Output median	Turns	Tool calls	Wall s	Cost $
C	12/12	201,029	1,974	8	8	25	1.30
Ada	12/12	226,474	2,929	9	8	34	1.52
Rust	12/12	200,394	1,992	8	7	26	1.32
Java	12/12	200,200	1,887	8	7	27	1.32
Python	12/12	230,900	2,476	8	8	30	1.39

Table 9: Each language over the four tasks, twelve runs. Cost is the sum.

Per run the median context was 202,386 tokens (163,394 to 324,890), median output 2,078 (912 to 4,854), 6 to 12 turns, 5 to 11 tool calls. Every run read the source file first; the median run read three files (source, `README.md`, `check.sh`), ran the shell two to four times and made one or two edits. In every T1 run the agent also updated `README.md` and `check.sh`. The 60 runs cost $6.85 and 29 minutes: $0.11 and 29 s a run.

Where the context goes: the first API call of every run shows the model 30,920 to 30,969 tokens (default system prompt, tool schemas, task), and each later call adds a median 926. The source file at o200k is 478 to 1,373 tokens; shown on every call it is at most 2.7% of a run's context at the median (Ada 4.5%, Python 1.6%). Context per run is turns times a harness constant, to within a few percent.

Between languages: C, Rust and Java share a median of 200k to 201k; Ada 226k and Python 231k, 13 to 15% more. Within a cell, three repetitions spread by a median 34,652 tokens and up to 107,310, so the between-language differences are of that order. The rank changes with the task: C is cheapest on T1, second on T2 and T3, third on T4; Rust is cheapest on T3 and costliest on T1. Ada costs the most output (2,929 median against 1,887 to 2,476), which is verbosity on the writing side, its reading side being 4.5%

of context. Python's T2 is the costliest T2 (240k context, 4,027 output, 47 s): its scan has two paths and both had to change.

7.3 What the pilot does and does not show

With retrieval at its floor and tasks this small, representation does not show in the cost. The agent read the file once at 1 to 4% of its context and spent the rest on the harness constant times its turns. The five are separated only by turns taken and tokens written, and at three repetitions that separation sits inside the run-to-run spread. This is the null the design allows for, at the one scale where it was always likeliest.

Each repository is one file, so retrieval is one `Read`: the pilot measures reconstruction and modification with retrieval held at its floor, the condition the design calls "highly relevant context given". Three repetitions bound the spread; they do not support a test between languages. One model was run. The bench is 25 to 133 lines, and nothing here scales.

8 Threats to validity

One author. Every set measured is written or shaped by the same person, whose C style is the one under study, and the `linearr` twin shows a port inherits the original's function shape. Nothing here separates "the C style" from "this author"; independent implementations of the equivalent sets are required for that. Section 6 answers it for the corpus measurements only.

Fifteen years. The genomics toolkits carry 2009 and 2011 in their file headers and no Java commit after their 2015 import; the C corpus is 2026, `bpnn` reaching back to 2015. Era, domain and language all differ between the groups, so their lower tail is not attributable to the language on this evidence. It does rule out the CCN 253 tail being a constant of the author.

Deliberately terse bench. `bench.c` packs 28 tokens per line and `Bench.java` is commented "deliberately terse". The bench measures the author at his densest, and its LLM-token ratios do not transfer to ordinary density, as `mincdp` and `linearr` show.

Python's two paths. The Python bench prints an idiomatic and an explicit row per operation, so a task touching the scan must change both. That is a different task shape, and the occurrence-versus-line difference limits the fixtures.

Lexical feature counts. The C++ rates of Table 7 are regex counts over comment-stripped source, not semantic counts. A `template` in a comment-free string literal would be counted; a template instantiated without the keyword nearby would not.

Tokenizer proxy. o200k is not the model's tokenizer.

Task authorship. The four tasks were written by the paper's author after seeing the code. They are behaviour-specified and identical across languages, which removes the location hint but not the choice of which behaviours to ask for.

Model and version. One model at one date. Agent cost is a property of the pair (representation, model).

Equivalence. Established by the repositories' own checks and this paper's fixtures, on the domains they cover. The `mincdp` demo was not re-run.

9 The experiment this motivates

The pilot fixes the harness. What follows varies what it held fixed, in two stages: a cheap one isolating the cost of a single indirection, and a larger one asking whether retrieval erases it.

Stage one, one pair per feature. Porting `ais` to C++ would give one number with every feature confounded in it, so the unit of analysis is the indirection. Six pairs, each a small C program and the same behaviour in C++ using one construct, the sixth at a realistic mixture: direct call against virtual override, `void *` or duplication against template, named function against overloaded operator, explicit initialise

and free against constructor and destructor, struct embedding against inheritance, and all together. Each pair passes one hidden suite, so equivalence is checked and not asserted.

The pilot found cost to be a harness constant times turns with the source under 3% of context, so a large C++ file shows nothing a small one does not. What must be long is the resolution path, not the file: the construction site in one file, the override in another, the instantiation in a header. Seven artifacts can be seven short files, and the effect is predicted in turns.

Rates come from Table 7, the mixed pair built at systems rather than header-only-library rates, so the C++ side is not a straw man and the choice is reported. Four tasks per pair, behaviour-specified with hidden tests, at least two requiring a change whose effect is not visible at the site of the change; a local task will show nothing, for the reason the pilot measured. The code lives with this paper beside `pilot/`, self-contained with its own build, kept out of `ais` so that nothing written for measurement enters that repository's build or dependency surface. Six pairs at two conditions, four tasks and five repetitions is 240 runs, about $26 and under two hours at four in parallel.

What the runs yield is a cost per unresolved edge. Combined with the function density and feature rates of Section 6, that gives the cost of a codebase at a given size and density: a model with measured parameters and a stated assumption about how they combine, not a measurement of a port nobody wrote.

Stage two, scale, retrieval and a framework. Stage one holds retrieval at its floor, which is what makes it cheap and what limits it. Stage two uses a repository where retrieval is not one `Read` (the full `ais/c`, 10,597 code lines) crossed with two retrieval conditions, ordinary tools and associative retrieval through `ais` itself. If better retrieval closes the gap between representations, the cost was retrieval; if both get cheaper and the gap stays, representation costs reconstruction on its own. At least 20 tasks, generated by rule before any run: local, cross-module and debugging, each behaviour-specified with hidden tests.

The framework rung belongs here as its own condition: one behaviour built on a framework that supplies part of it, against the same behaviour built without one. A language abstraction hides code that is still in the repository; a framework hides control flow that is not in the repository at all, since an annotation, a naming convention or a container decides what runs and the agent can read it nowhere in the tree. One endpoint of the System A kind is enough: about 200 lines of behaviour, built three ways, judged at the HTTP boundary.

Common to both. Every condition implements one specification, so the source-token difference between conditions is the syntactic overhead, measured and not estimated, recorded on both sides: tokens read to reach the point of change, tokens written to make it. Ceremony costing a person one habitual gesture is paid by an agent at full price in both directions, and a representation can be compact to read and expensive to edit.

Three outcomes, all already recorded by the pilot harness. *Tokens*: context to a hidden-test pass, read and written separately. *Time*: wall seconds to a passing run. *Debugging*: turns and tool calls to the pass, failed attempts, and regressions in behaviour the task did not name. Failures are reported and never dropped, so success probability is an outcome and not a filter.

Runtime performance is not an outcome. Between C and C++ it is within noise, which is what makes the pair a clean instrument: the abstraction is free at run time, so anything it costs is paid elsewhere. Between a bare stack and a framework stack the difference is real and large and has been measured by others on identical hardware against a fixed specification; this paper cites that work rather than reproducing it.

Repetitions of one task are not independent. Mixed-effects regression on log context with task and model as random effects, effect sizes and intervals, per-task distributions printed. The pilot resolved nothing below 15% of median context at three repetitions, so the design carries five and each condition reports which effect sizes it could not have seen.

H_0: after task, model, size and retrieval, representation has no effect. H_1: reconstruction cost, measured on the semantic graph (relationships traversed, plausible targets, dispersion across files), predicts agent cost beyond token count. H_2: some abstractions raise it and some lower it; RAII and standard containers are the candidates for lowering, and framework control flow for raising it most, being

the one form of hiding that puts the answer outside the repository.

9.1 Prediction, recorded before the rungs exist

No C++ implementation of any of these behaviours existed on 24 August 2026, when this was written. The direction is recorded now so it cannot be fitted to the numbers later, with the results that would refute it.

The prediction concerns tokens, wall time and the turns a change takes to get right. The mechanism is not token count. Reading more source costs an agent roughly linearly and cheaply: the file under maintenance was at most 2.7% of context at the median, so doubling its length adds about 2.5% to a run. Resolving a relationship costs a search over the targets a call site could reach, and its failure mode is not slowness but an edit that compiles, passes the build and is wrong. Explicitness is predicted to pay where the second cost applies and to be nearly free where only the first does.

1. **Local tasks**, the behaviour to change visible at the site of the change: no advantage to the explicit rungs, and a possible advantage to the abstract ones, since RAII removes a lifetime obligation the C rungs carry on every path out of a function. Predicted loser: explicit C.
2. **Cross-module and debugging tasks**, where the answer depends on a relationship not visible at the site of the change: the explicit rungs cost less context and fail less often, the gap growing with plausible targets per call site rather than with source size.
3. **The mix**: an effect smaller than either, in whichever direction the task proportions decide. The primary analysis is per task type; an aggregate alone would hide both.
4. **The framework condition** costs the most of any rung on cross-module tasks, because the control flow deciding the answer is not in the repository in any form.

Table 10 gives the path an agent must walk to establish what one call does, for one behaviour: fetch a record and return it.

	Path from the call to the behaviour	Steps	No file
C, as written	caller, `ais_find`, the store read	3	0
C++, abstract	interface, construction site, override, template instantiation, constructor and destructor, operator overload, allocator	7	3
Java, as written	servlet, verb class, DAO returning the response body	3	0
Java, framework	controller, service, repository interface, its generated implementation, entity, mapper, DTO, with the transaction decided by a proxy	7	3

Table 10: Predicted path length for one behaviour, in two languages, at each end of the axis. *Steps* counts the artifacts an agent must resolve to establish what the call does. *No file* counts the steps whose behaviour has no textual origin anywhere in the repository. These are predicted values; measuring them is what the rungs are for.

The second column is what people argue about, and layered designs are defended on it fairly. The third has no answer for an agent, being behaviour that cannot be read at any price: a template instantiation, an implicit destructor, a generated repository implementation, a transaction boundary that exists only as an annotation. It is zero at both short paths and non-zero at both long ones, in two languages that share nothing else. The prediction is that agent cost tracks the third column rather than the second, and that the two pairs move together.

Three results refute the prediction as stated. The explicit rungs costing more context, or failing more often, on cross-module or debugging tasks. The gap between rungs closing when retrieval improves, which would mean the cost was retrieval and representation only a proxy for it. Tokens written on the explicit rungs rising with the repetition explicitness requires, by enough to cancel the reading advantage, which would make the answer a property of the task mix rather than of the representation.

An effect below 15% of median context is not resolvable at three repetitions: Ada and Python sat 13 to 15% above the other three and that difference was inside one cell's spread. The experiment needs enough repetitions to resolve less than that, or it will report a null it cannot distinguish from this one.

10 Reproducibility

In this directory: `measure.py` (every source number; run it, and the commits it read are in its JSON), `tables.py` (the table bodies above, from the JSON), and `pilot/` with `mkfix.py` (the fixtures, seeded), `tasks.py` (the four task texts), `langs.py` (build, run and bug substitution per language), `setup.py` (templates, baseline and bug check), `run.py` (the harness), `hidden.py` (the hidden tests), `summarize.py`, and `runs/` with every run's `result.json`, `final.diff`, `hidden.log` and full `trace.jsonl`. The C and C++ pairs of Section 9, when written, go beside `pilot/` in this directory, with their own build and their own checker.

Toolchain: gcc 13.3.0, GNAT 13.3.0, rustc 1.75.0, OpenJDK 21.0.11, CPython 3.12.3, cloc 1.98, lizard 1.24.0, tiktoken 0.14.0, libclang 18.1.1, Claude Code 2.1.241, model `claude-sonnet-5`, Linux 6.8.

11 What is open

The measurement this paper is built for is absent: no C++ and no framework implementation of any behaviour here exists. Stage one of Section 9 is 240 runs, about $26 and two hours, and it is what turns the prediction into a result.

Two things this paper establishes do not depend on that. The indirection whose cost is in question is real and large in released code by other authors: 60.9 functions per thousand lines against 28.4, at a median complexity of 1. And the long path is not required to deliver the behaviour, which one production system of 39,621 lines and 37 tables shows by existing.

One thing it establishes is negative and holds regardless: at one-file scale a coding agent's cost is a harness constant times its turns, the representation under 3% of what it reads, so a one-file language comparison cannot answer this question at all.

References

[1] V. Gavrilov. Repositories. `https://github.com/Anode1`.

[2] V. Gavrilov. *ais: An extension of your associative memory*. `https://github.com/Anode1/ais`, commit 84e3ad5.

[3] V. Gavrilov. *ais development style guide*. `https://github.com/Anode1/ais/blob/main/doc/dev/STYLE.md`.

[4] V. Gavrilov. *Five languages, two core operations, one algorithm*. `https://github.com/Anode1/ais/blob/main/tests/perf/LANG_COMPARISON.md`.

[5] V. Gavrilov. *mincdp: a minimal Chrome DevTools Protocol client in C and Java*. `https://github.com/Anode1/mincdp`, commit 1bebbe3.

[6] V. Gavrilov. *linearr: least squares in C*. `https://github.com/Anode1/linearr`, commit 0a79370.

[7] MISRA. *MISRA C:2012, Guidelines for the use of the C language in critical systems*. MIRA Ltd., 2013.

[8] A. Danial. *cloc: Count Lines of Code*, version 1.98. https://github.com/AlDanial/cloc.

[9] T. Yin. *lizard: a simple code complexity analyser*, version 1.24.0. https://github.com/terryyin/lizard.

[10] OpenAI. *tiktoken*, version 0.14.0. https://github.com/openai/tiktoken.

[11] Anthropic. *Claude Code*, version 2.1.241, headless mode. https://docs.claude.com/en/docs/claude-code.